Sistemas Operativos I

Trabajo Práctico 4 - threads y procesos

Objetivos

- Analizar diferencias entre **procesos, y threads.**
- **API de C POSIX para threads y semáforos.**

Referencias

[1] Tanenbaum, Bos – Modern Operating Systems - Prentice Hall; 4 edition (March 10, 2014) - ISBN-10: 013359162X
[2] Douglas Comer - Operating System Design - The Xinu Approach. CRC Press, 2015. ISBN : 9781498712439
[3] Silberschatz, Galvin, Gagne - Operating Systems Concepts - John Wiley & Sons; 10 edition (2018) – ISBN 978-1-119-32091-3

Software y Hardware

Este trabajo práctico se realiza en su totalidad en UNIX/Linux.

Ejercicio 1. *Sistemas Operativos derivados de BSD.*

a. Describa con 3 oraciones cada uno de los siguientes sistemas operativos : NetBSD, OpenBSD, FreeBSD.
b. Presente ejemplos "famosos" donde se utilice FreeBSD y OpenBSD.
c. ¿Por qué se dice que OpenBSD ha colaborado ampliamente con proveer un sistema operativo y herramientas para sistemas seguros? ¿Por qué querrían alguien utilizar un sistema "mas seguro"? (¿no son seguros Linux y Windows?).
d. Las páginas de Wikipedia de cada sistema operativo en a. dicen en su primera oración que cada uno de estos sistemas es Libre y Open Source.
   d.1. ¿Tan importante es eso que lo ponen primero antes que nada?.
   d. 2. Explique qué significa ambos conceptos y si están relacionados o no.
e. ¿Qué utilidad tiene que un sistema operativo sea Libre?. Al fin y al cabo, si necesitara portar o adaptar Windows a un televisor simplemente lo podría modificar uno mismo, o pagarle a Microsoft. Critique esta frase (1).

Ejercicio 2. *Funciones de POSIX para threads y semáforos.*

a. Estudie la interfaz de llamadas a funciones de C POSIX para threads y semáforos, y explique que realiza cada función.
   Semáforos: sem_open, sem_wait, sem_post, sem_close, sem_unlink.
   Threads: Tipo de datos pthread_t, pthread_create, pthread_join.
   Para leer desde la documentación oficial en el sistema puede leer la página de manual de cada función. Por ejemplo, ejecutando en una terminal+shell:
   man sem_open
b. ¿Windows, al programar en Java o C, implementa threads a nivel del kernel o en espacio de usuario como parte del entorno en tiempo de ejecución?. ¿Y en macOS y GNU/Linux?

Ejercicio 3.

Portar el programa del Ejercicio 3. del tp3 (portar la implementación de mutex) utilizando threads y semáforos en Linux (operar e incrementar serán threads).

(1) Criticar: Analizar pormenorizadamente algo y valorarlo según los criterios propios de la materia de que se trate. (Diccionario de la Real Academia Española).

Ejercicio 4.

a. Escriba un programa en GNU/Linux que genere 3 procesos que utiliza la función capicuá más debajo. Deben realizar lo siguiente:
   El primer proceso busca capicúas entre 0 y 10000.
   El segundo proceso busca capicúas entre 10001 y 20000.
   El tercer proceso busca capicúas entre 20001 y 30000.
b. Escriba un programa en GNU/Linux que utilice 3 threads para realizar la misma función que el inciso a.
c. Utilizando el comando **`time`** de UNIX observe cuánto demora la ejecución de cada programa. Si los tiempos son similares, o incluso si el inciso a. presenta mejor tiempo, ¿por qué se dice entonces que los threads "son mas livianos o tienen mejor performance"?

```
/* función que devuelve un valor distinto de cero si n es capicúa */
int es_capicua(int n) {
    int original = n;
    int invertido = 0;
    int resto;

    while (n > 0) {
        resto = n % 10;
        invertido = invertido * 10 + resto;
        n = n / 10;
    }

    return original == invertido;
}
```

(1) *Criticar:* Analizar pormenorizadamente algo y valorarlo según los criterios propios de la materia de que se trate. (Diccionario de la Real Academia Española).